EX 2: MULTI-LAYER PERCEPTRON (MLP) FOR CLASSIFICATION

AIM

To develop a Multi-Layer Perceptron (MLP) for classifying Iris flowers using the Iris dataset, experiment with different hidden layer configurations and activation functions, and evaluate the model using accuracy and loss.

PROCEDURE

1. Import the required libraries, including NumPy, Matplotlib, TensorFlow/Keras, and scikit-learn.
2. Load the Iris dataset containing four flower features and three species classes.
3. Convert the target labels into one-hot encoded vectors and standardize the input features.
4. Split the dataset into training and testing sets using an 80:20 ratio.
5. Define an MLP model with configurable hidden layers and activation functions.
6. Compile the model using the Adam optimizer, categorical cross-entropy loss, and accuracy metric.
7. Train and evaluate the model using different hidden layer configurations and activation functions such as ReLU, tanh, and sigmoid.
8. Visualize the training and validation accuracy and loss curves.
9. Accept new flower measurements as input, preprocess them, and predict the corresponding Iris species.

CODE

```
import matplotlib.pyplot as plt

import numpy as np

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler, OneHotEncoder

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, Input


# Load and preprocess the dataset

iris = load_iris()

X = iris.data

y = iris.target.reshape(-1, 1)

classes = iris.target_names


encoder = OneHotEncoder(sparse_output=False)

y_encoded = encoder.fit_transform(y)


scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_encoded, test_size=0.2, random_state=42
)

# Create MLP model
def create_mlp(input_dim, output_dim, hidden_layers,
               activation='relu'):
    model = Sequential()
    model.add(Input(shape=(input_dim,)))

    for units in hidden_layers:
        model.add(Dense(units, activation=activation))

    model.add(Dense(output_dim, activation='softmax'))

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )
    return model

# Experiment with different configurations
hidden_layer_configs = [[8], [16, 8], [32, 16, 8]]
activations = ['relu', 'tanh', 'sigmoid']

last_model = None
last_history = None

for hidden_layers in hidden_layer_configs:
    for activation in activations:
```

```
print(
    f"\nHidden Layers: {hidden_layers}, "
    f"Activation: {activation}"
)

model = create_mlp(
    input_dim=4,
    output_dim=3,
    hidden_layers=hidden_layers,
    activation=activation
)

history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=5,
    verbose=0,
    validation_split=0.1
)

test_loss, test_acc = model.evaluate(
    X_test, y_test, verbose=0
)

print(f"Test Accuracy: {test_acc:.4f}")
print(f"Test Loss: {test_loss:.4f}")

# Plot accuracy and loss
plt.figure(figsize=(10, 4))
plt.suptitle(
    f"Hidden Layers: {hidden_layers}, "
    f"Activation: {activation}"
)
```

```
        plt.subplot(1, 2, 1)
        plt.plot(history.history['accuracy'], label='Train Accuracy')
        plt.plot(history.history['val_accuracy'], label='Val Accuracy')
        plt.xlabel('Epoch')
        plt.ylabel('Accuracy')
        plt.title('Model Accuracy')
        plt.legend()

        plt.subplot(1, 2, 2)
        plt.plot(history.history['loss'], label='Train Loss')
        plt.plot(history.history['val_loss'], label='Val Loss')
        plt.xlabel('Epoch')
        plt.ylabel('Loss')
        plt.title('Model Loss')
        plt.legend()

        plt.tight_layout()
        plt.show()

        last_model = model
        last_history = history

# Predict a new Iris flower
sepal_length = float(input("Sepal length (cm): "))
sepal_width = float(input("Sepal width (cm): "))
petal_length = float(input("Petal length (cm): "))
petal_width = float(input("Petal width (cm): "))

user_input = np.array([[
    sepal_length, sepal_width,
    petal_length, petal_width
]])
```

```
user_input_scaled = scaler.transform(user_input)

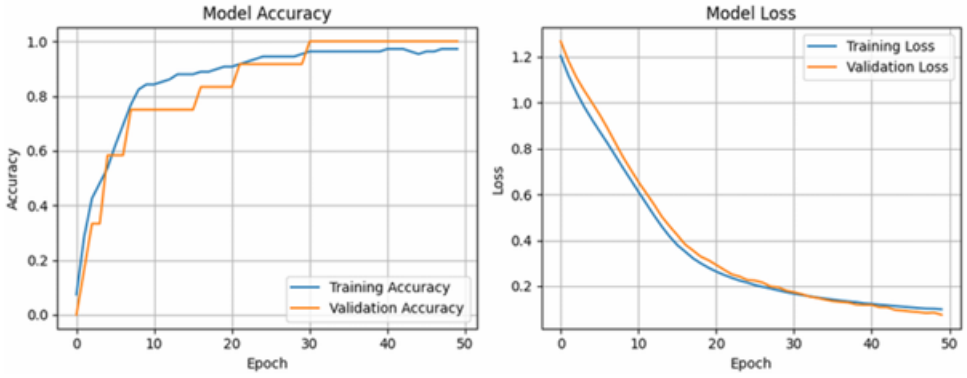

prediction = last_model.predict(user_input_scaled, verbose=0)
predicted_class_index = np.argmax(prediction)
predicted_class_name = classes[predicted_class_index]


print(f"\nPredicted Iris Species: {predicted_class_name}")
```

OUTPUT

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 40, 64)	2,496
lstm (LSTM)	(None, 96)	61,824
dense (Dense)	(None, 39)	3,783

```
Total params: 68,103 (266.03 KB)
Trainable params: 68,103 (266.03 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/3
391/391 ━━━━━━━━━━━━━━━━━━━━ 58s 142ms/step - loss: 2.6905
Epoch 2/3
391/391 ━━━━━━━━━━━━━━━━━━━━ 58s 147ms/step - loss: 2.2819
Epoch 3/3
391/391 ━━━━━━━━━━━━━━━━━━━━ 57s 146ms/step - loss: 2.1622
```



RESULT

Thus, a Multi-Layer Perceptron (MLP) was successfully implemented using TensorFlow/Keras to classify Iris flowers. Different hidden layer configurations and activation functions were evaluated using accuracy and loss. The training performance was visualized, and the trained model was used to predict the species of a new Iris flower.